Space–Time Trade-offs in Graph Traversal

An Empirical Study of BFS, DFS, and Graph Representations

- Student: **Qiyun Ge**
- Course: 420-A10-AS C2 — Introduction aux structures de donnée
- Instructor: Ali Moridnejad
- April 2026

1. Objective

The goal of this project is to investigate the time–space trade-offs in graph traversal algorithms under different system and representation settings.

Specifically, we aim to analyze:

- The impact of graph representation:
  - adjacency list vs adjacency matrix
- The behavior of traversal algorithms:
  - Breadth-First Search (BFS)
  - Depth-First Search (DFS)
- The influence of runtime environment:
  - Python vs C++

We measure two key aspects:

- **Time complexity (empirical runtime)**
- **Space usage (peak frontier size during traversal)**

The objective is not only to compare performance, but also to understand how design choices (representation, algorithm, runtime) affect computational efficiency under varying graph densities.

2. Methodology

2.1 Graph Model

We use randomly generated graphs of fixed size:

- Number of nodes: n = 1000，5000，10000
- Graph type: Random graph G(n, p)
- Density parameter: p ∈ [0.05, 0.2,0.5,0.8]

The parameter p controls the edge density:

- Small p → sparse graph
- Large p → dense graph

```
# edge generate process of random graph
        for u in range(n):
            for v in range(n):
                if u != v and rng.random() < p:
                    edges.append((u, v))
```

2.2 Graph Representations

Two common graph representations are evaluated:

- **Adjacency List**
  - Stores only existing edges
  - Efficient for sparse graphs
- **Adjacency Matrix**
  - Stores all possible edges
  - Requires $O(n^2)$ space

2.3 Traversal Algorithms

We compare two fundamental traversal strategies:

- **Breadth-First Search (BFS)**
  - Explores nodes level by level
  - Typically uses a queue
- **Depth-First Search (DFS)**
  - Explores along a path before backtracking
  - Typically uses a stack (or recursion)

2.4 Runtime Environments

All experiments are executed in two environments:

- Python implementation
- C++ implementation

This allows us to compare not only algorithmic behavior, but also the impact of language-level performance differences.

2.5 Measured Metrics

For each configuration, we record:

- **Runtime (seconds)**
  Total execution time of the traversal
- **Peak Frontier Size**
  Maximum number of nodes held in the frontier (queue/stack) during traversal
- **Visited Nodes Count**
  Used for sanity checking (should be equal to n for connected graphs)

2.6 Theoretical Expectations

This section summarizes the expected behavior of graph traversal algorithms based on theory.

---

1. BFS vs DFS Runtime

Both BFS and DFS have time complexity:

O(V + E)

- each node is visited once
- each edge is processed a constant number of times

Expectation:
BFS and DFS should have **similar runtime performance**

---

2. Graph Representation

- **Adjacency List**
  - O(V + E), iterate only neighbors
- **Adjacency Matrix**
  - $O(V^2)$, scan full row

Expectation:

- List is faster in sparse graphs
- Gap decreases in dense graphs

---

3. BFS Space Behavior

- DFS: stack size ≈ depth → small

- BFS: queue stores frontier → large

In G(n, p):

- degree ≈ p·n
- higher density → faster expansion

Expectation:

- BFS frontier increases with density
- may approach O(n)
- DFS remains stable

---

4. Python vs C++

- Python: higher overhead
- C++: compiled, efficient memory

Expectation:

- C++ is consistently faster
- difference larger for adjacency list

Summary

Factor	Expected Outcome
BFS vs DFS	Similar runtime
BFS vs DFS (space)	BFS >> DFS
List vs Matrix	List faster
Density ↑	BFS frontier ↑
Python vs C++	C++ faster

2.7 Experimental Workflow

Each experiment follows the pipeline:

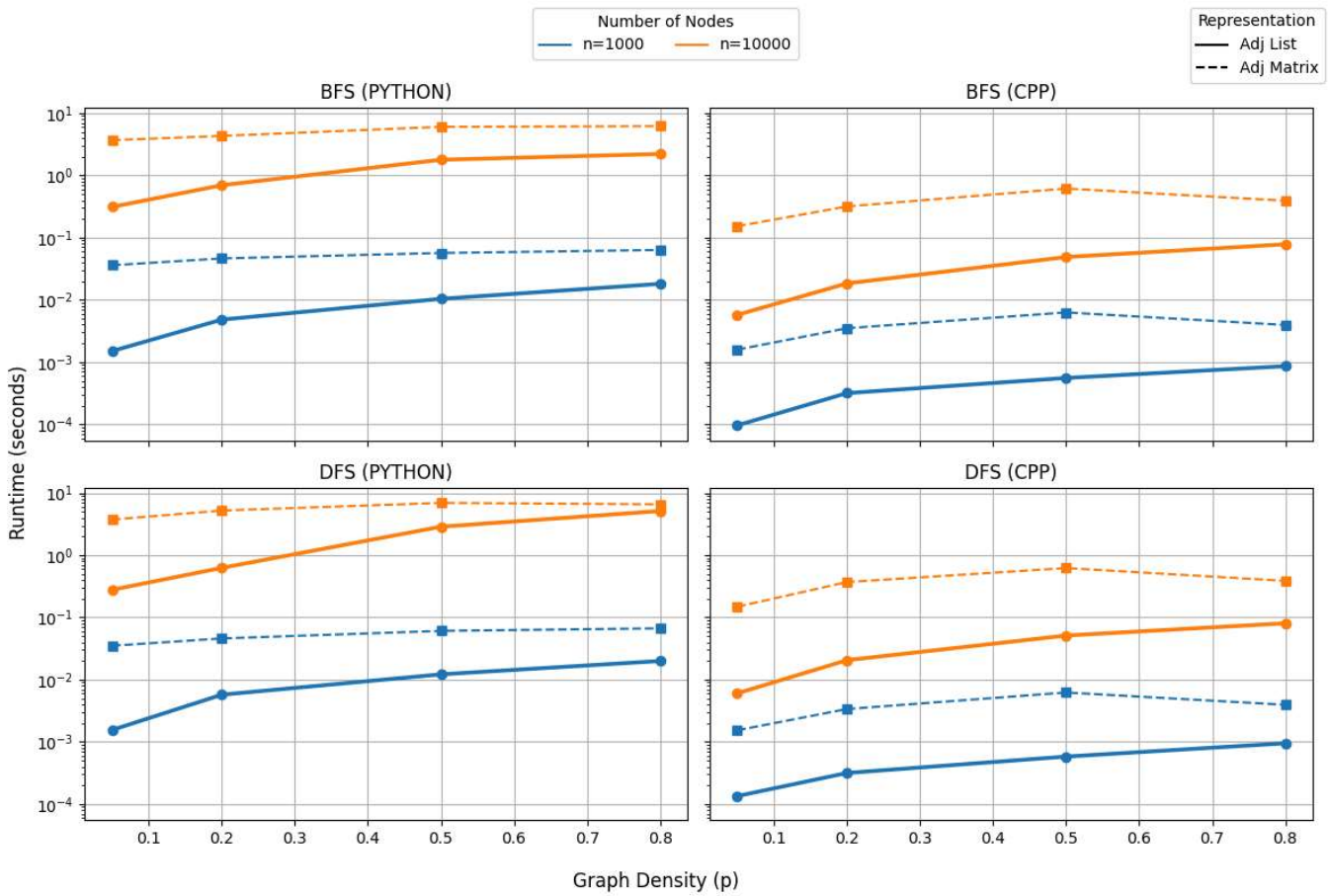
1. Generate a graph with given parameters (n, p)
2. Construct the chosen graph representation
3. Run the traversal algorithm (BFS or DFS)
4. Measure runtime and peak frontier
5. Record results into a CSV file

The collected dataset is then analyzed in this notebook.

3. Experimental Analysis

This section compares experimental results with the theoretical expectations.

3.1 BFS vs DFS Runtime



Observation:

- BFS and DFS show very similar runtime across all settings

Explanation:

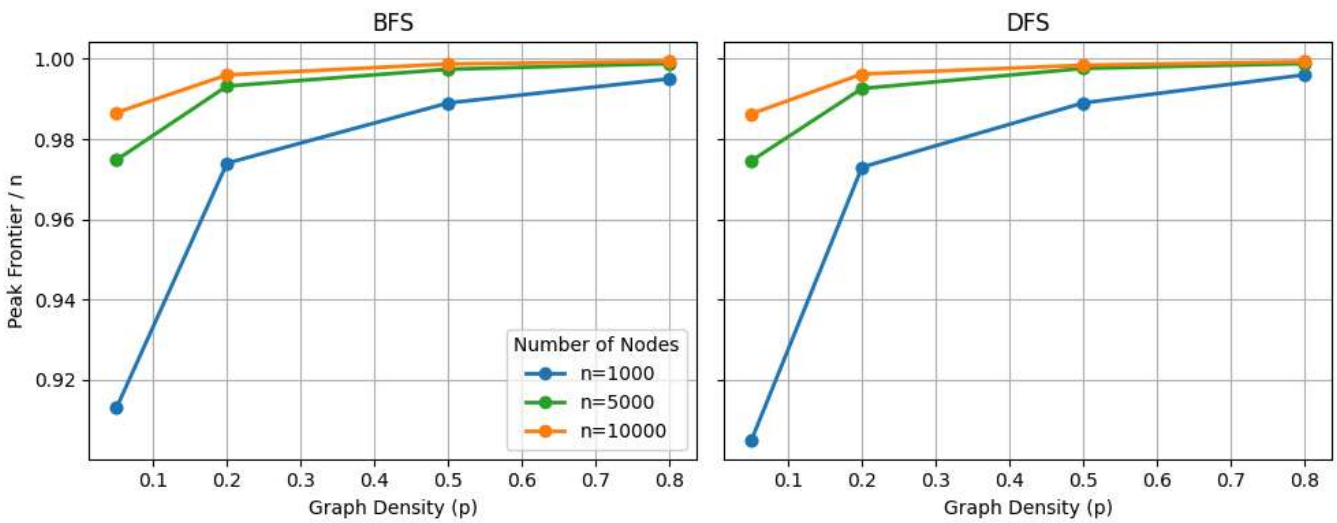
- Both have complexity O(V + E)
- traversal order does not significantly affect total cost

Conclusion:

BFS and DFS have **comparable runtime performance**

3.2 Peak Frontier Comparison

We analyze peak frontier normalized by graph size.



Observation:

- Both BFS and DFS show similar peak frontier ratios
- The frontier rapidly approaches 1.0 as graph density increases
- This behavior is consistent across different graph sizes

Explanation:

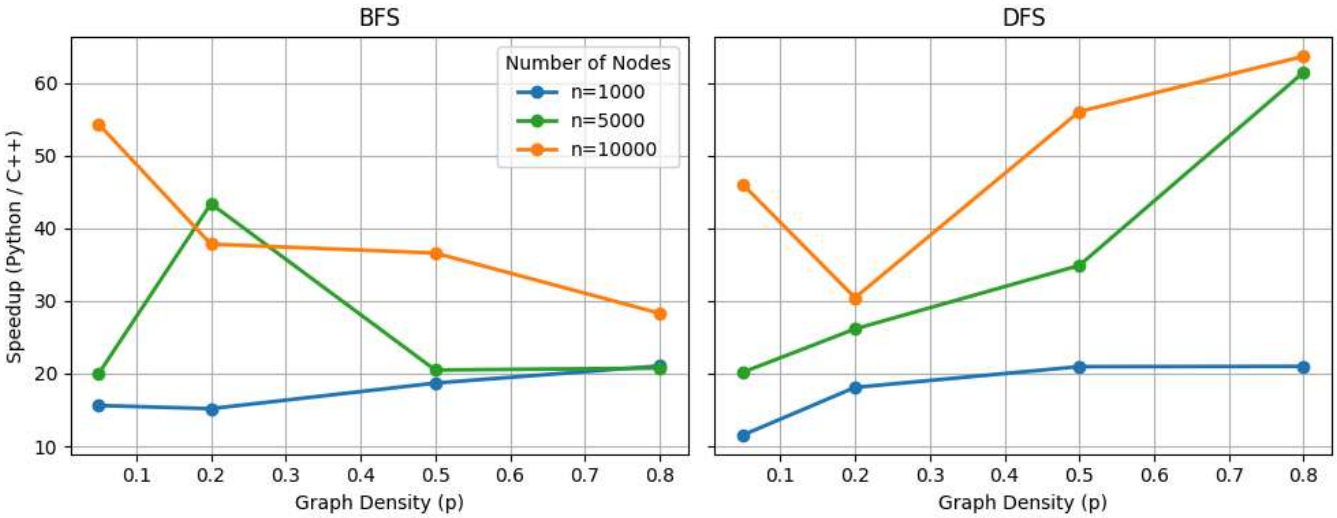
- In G(n, p), higher density leads to larger expected degree (≈ p·n)
- The graph becomes highly connected, allowing traversal to quickly reach most nodes
- As a result, the number of discovered nodes rapidly approaches n
- The similarity between BFS and DFS indicates that, in dense random graphs, traversal strategy has limited impact on the maximum explored node set

Conclusion:

Peak frontier size is primarily determined by **graph density and connectivity**, rather than traversal algorithm or graph representation

3.3 Runtime Environment Comparison

We examine the performance gap between the Python and C++ implementations by analyzing the runtime ratio (Python runtime / C++ runtime). This allows us to determine whether the runtime environment changes only the constant factor or also affects the overall scaling trend.



Observation:

- C++ is consistently faster than Python

Explanation:

- Python: interpreter + dynamic typing
- C++: compiled + better memory efficiency

Conclusion:

Implementation language introduces **large constant-factor differences**

3.4 Measurement Noise

Observation:

- Small fluctuations appear in runtime results

Explanation:

- execution time is very small
- affected by system noise (CPU, cache, scheduling)

Conclusion:

Results are valid, but **averaging multiple runs would improve stability**

4. Conclusion

This project analyzed the time–space trade-offs of graph traversal under different representations and runtime environments.

Key Findings

- BFS and DFS have **similar runtime**, confirming their shared complexity O(V + E)
- Adjacency list **outperforms matrix**, especially in sparse graphs
- C++ provides **substantial speedup** over Python

Final Insight

Performance is primarily determined by:

- **data representation** (list vs matrix)
- **execution environment** (Python vs C++)

rather than the choice between BFS and DFS.

**Limitation*

- Runtime measurements are affected by noise due to very small execution time
- More reliable results require repeated runs and averaging

Takeaway

> In practical systems, data structures and implementation choices often matter more than asymptotic algorithm differences.